多线程笔记整理

一、 概述

1) 网络应用之间的通信一般需要明确 IP 和端口，因为一个网络应用往往占用一个或多个端口，然后采取某种网络协议来完成通信。
2) 网络编程一般使用 socket 类又名套接字，以 BIO 或 NIO 的形式，常用 BIO，NIO 常用于线程并发的编程中
3) 常见的协议及区别

   1). UDP 协议

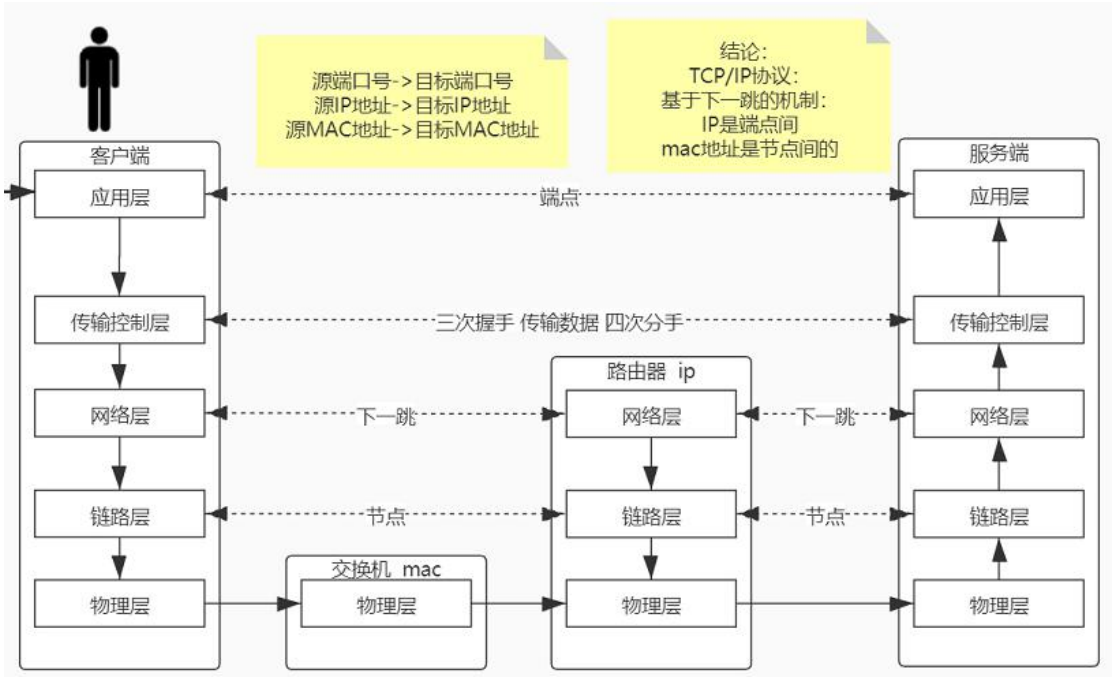
   传递数据时不关心连接状态直接发送,他的传输效率高,传递的数据容易丢包

   2). TCP/IP 协议

   传递数据时关系连接的状态,连接成功才会发送数据,他的传输效率相对于 UDP 协议要低,会通过三次握手机制保证数据的完整性，java 编程默认使用的协议

   3). Http 协议

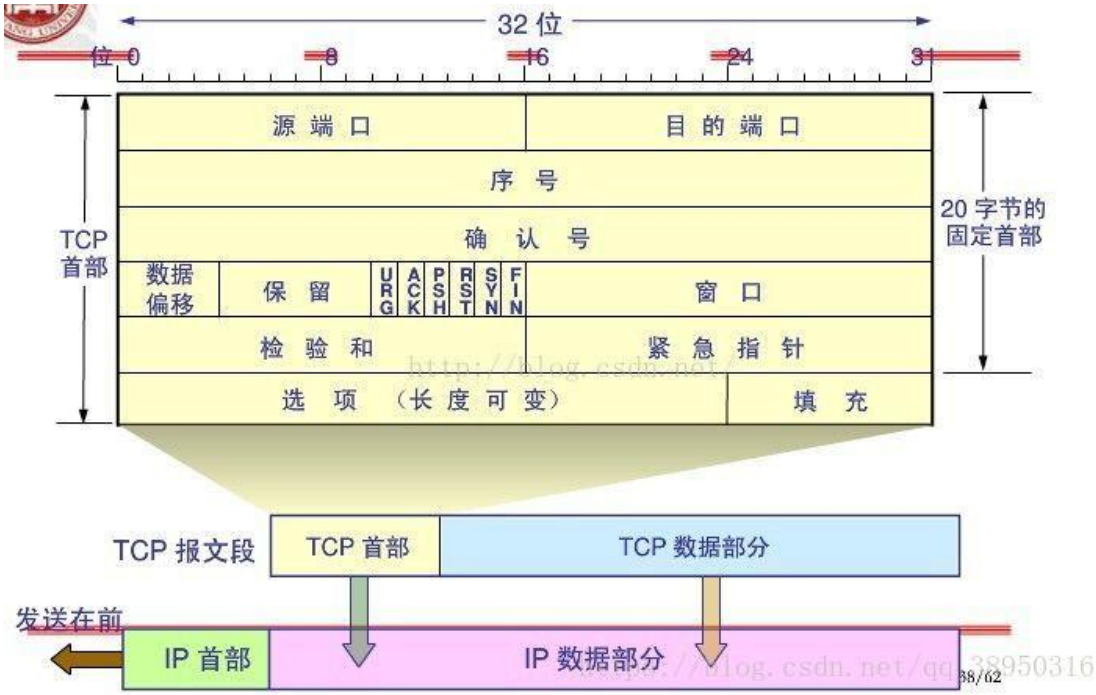
   相对 TCP 协议效率较低，是对 TCP 协议的封装
4) http/socket/tcp-ip 通信详解（重点）
   a) 网络传输的七层协议是概念，实现的时候将其合并为以下 5 层（这五层要被下来）。
   b) 最上层的应用层是可以实现多个自定的协议（如 http 协议，ftp 协议等），所有上层的协议都是对下层协议接口的调用实现的。如传输控制层有 TCP 和 UDP，http 其实就是对 TCP 的封装。



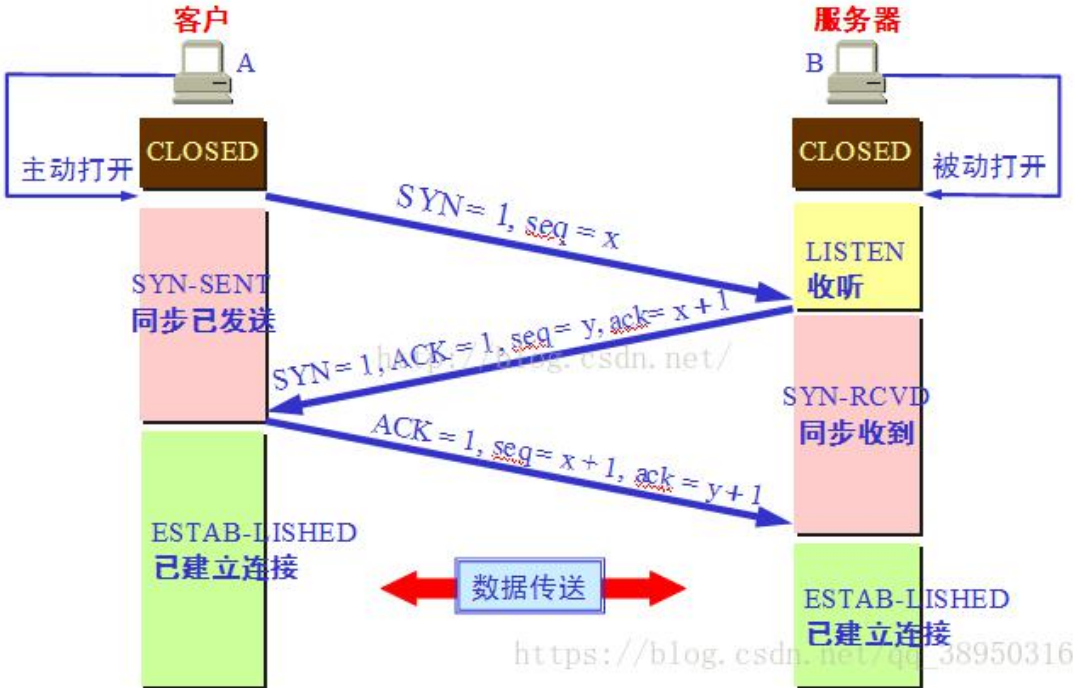
   a) HTTP 以及 socket 通信过程分析
   b) 首先我们在发起一个 HTTP 请求时，应用层会用 socket 对象来包装我们的请求信息，然后调用传输控制层的 TCP/IP 协议接口，与服务端发生三次握手确认连接可用。
   c) TCP 的三次握手

[1] 首先就是 TCP 包的三个重要的标志符（只有 0/1 两种状态）

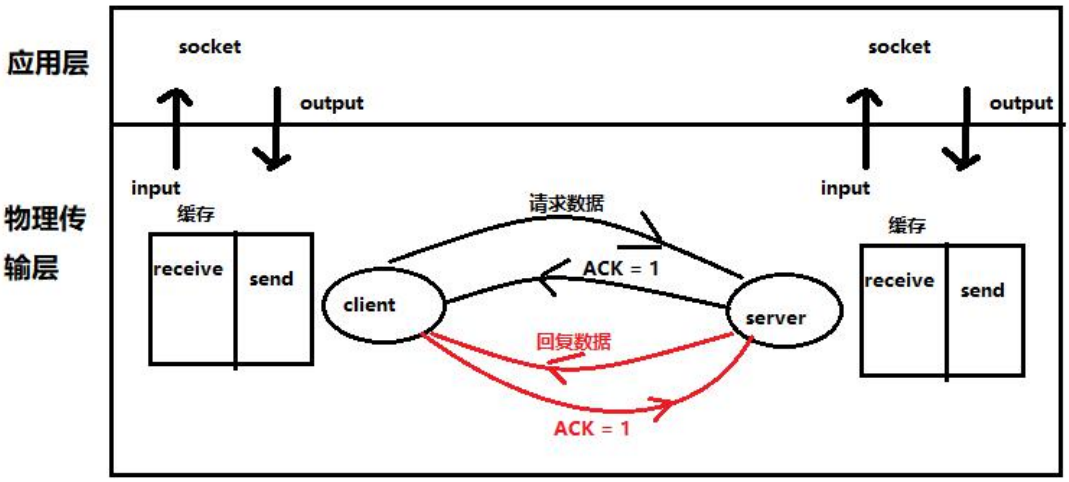
- ACK:确认是否有效
- SYN:请求建立连接
- FIN:希望断开连接



[2] 客户端传输控制层会打一个包，将 SYN 位置为 1，请求链接；服务端会收到这个包，如果可以进行消息接受，就会回复一个包，将 SYN 置为 1，同时将 ACK 置为 1，表示同意建立链接；然后客户端又会回复一个包，将 ACK 置为 1，表示同意建立连接。三次握手执行完毕后，不管服务端是否已经 accept，双方就会在各自的内存中开辟一块缓存用来存放 receive 数据和 send 数据。



[3] 建立连接并开辟了缓存之后，就可以就行数据传输了。我们会把发出的消息和收到的消息用 socket 对象的输入流（InputStream）和输出流(OutStream)来包装,应用层的输出流（OutStream）会被压入开辟的缓存的 Send 队列中，由网卡发送给对方，同时会接受到对方发来的包，ACK 置为 1，表示确定收到了消息，对方发来的消息会存入缓存中的 receive 队列中，由 socket 对象取出，传至应用层（inputStream），这样的话就完成了数据的传输，其实就是向缓存中压入数据或者取出数据。



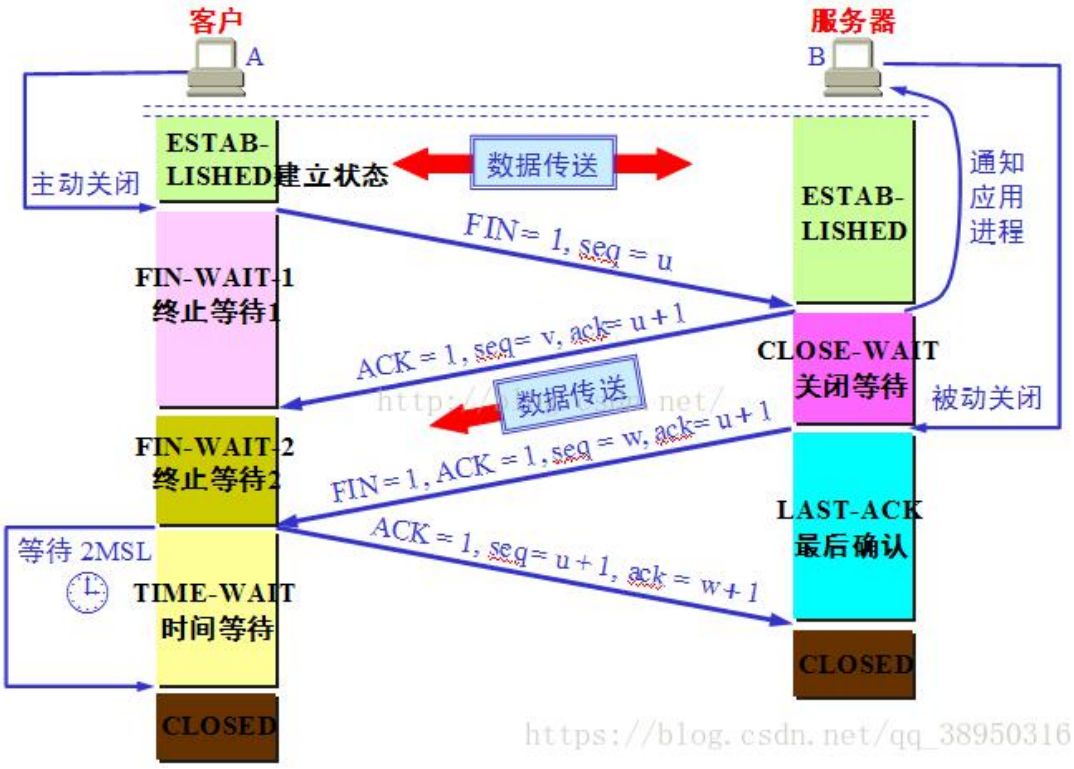
d) 当通信完毕后，应用层执行 socket 关闭方法，物理传输层会进行四次挥手来断开连接。

[1] 客户端向服务端发送一个包，将 FIN 置为 1，表示想断开连接；

[2] 服务端向客户端发送一个包，将 ACK 置为 1，表示确定收到了消息

[3] 服务端把剩余的业务处理完后，将 FIN 置为 1，ACK 置为 1，表示消息处理完了，可以断开连接；

[4] 客户端向服务端发送一个包，将 ACK 置为 1，表示收到了服务端的断开请求，链接断开，同时缓存清空。



e) 一些基础知识

[1] Socket 套接字是客户端 ip 和端口+服务端 ip 和端口组成的全球唯一的键值对，只要保证套接字的唯一，就能进行数据传输，也就是说一个服务端口可以对多个客户端进行服务，一个客户端口可以同时访问多个服务端。

f) 然后我们对下面几层进行一些说明（网络层，链路层，物理层）

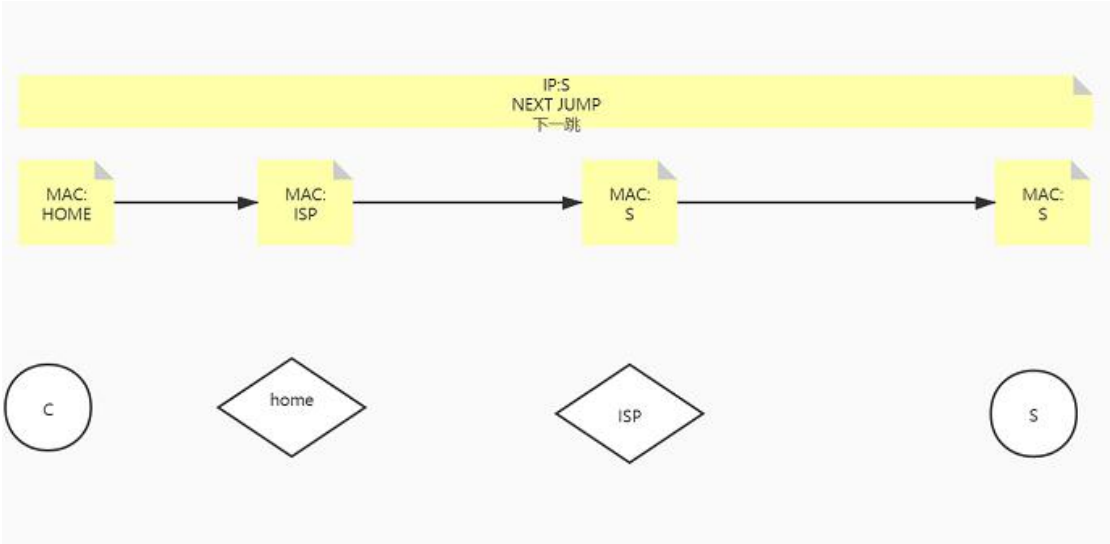
[1] 我们的因特网可以看作是由路由器，路由总站组成的一层又一层的局域网；

[2] 网络层做的是 IP，掩码等信息的分配，路由表的存储

[3] 链路层存着下一个路由器的信息，然后拿着网络层传来的 IP,目标地址，消息，根据路由表映射关系，通过物理层，把信息传递至下一个路由节点；

[4] 然后下一个路由节点，链路层又根据它的路由表映射关系，将消息传给下一个节点

[5] 直到链路层的地址和目标地址一致，才能完成消息的接受。



5) 我们可以通过 new ServerSocket + 端口的形式创建一个服务对象，通过 new Socket + 服务 IP,端口的形式创建一个连接对象，并通过输出流 OutputStream 向服务对象发送信息，通过捕获输入流 InputStream 来获取服务器的信息；
服务对象通过 accept 方法来获取连接对象；通过捕获连接对象中的输入流 InputStream 来获取，客户端的信息，通过输出流 OutputStream 向客户端发送消息。

```
public static void main(String[] args) throws IOException {
    //创建socket对象                                    创建连接对象
    Socket socket = new Socket( host: "192.168.43.81",  port: 8989);

    //向服务器发送数据
    OutputStream outputStream = socket.getOutputStream();
    outputStream.write("hello Server".getBytes());
    //代表客户端发送的数据完成
    socket.shutdownOutput();                            向服务器发送消息

    //获取服务器端响应数据
    InputStream inputStream = socket.getInputStream();
    int len = 0;
    byte[] b = new byte[1024];
    StringBuilder builder = new StringBuilder();
    while(true){
        len = inputStream.read(b);                      接受服务器消息
        if(len==-1) {break;}
        builder.append(new String(b, offset: 0,len));
    }
    System.out.println(builder.toString());

    socket.shutdownInput();//确定读取响应数据结束
```

```java
//创建一个服务器
ServerSocket serverSocket = new ServerSocket( port: 8989);
System.out.println("服务器已经启动........");
while(true) {
    //让服务器接受/接收客户端
    Socket socket = serverSocket.accept();

    //处理请求数据
    InputStream inputStream = socket.getInputStream();
    StringBuilder builder = new StringBuilder();
    int len = 0;
    byte[] b = new byte[1024];
    while (true) {
        len = inputStream.read(b);
        if (len == -1) {break;}
        builder.append(new String(b,  offset: 0, len));
    }

    System.out.println(builder.toString());
    socket.shutdownInput();//获取请求数据 结束
    //处理业务
    //服务端响应客户端数据
    OutputStream outputStream = socket.getOutputStream();
    outputStream.write("讲".getBytes());
    socket.shutdownOutput();
}
```

创建服务器

获取连接对象

获取连接对象中的消息

向连接对象发送消息

6) 在网络编程中，ServerSocket 如果没有并发操作，那么每次只有一个线程，必须等获取的套接字对象操作完，才能对下一个请求进行操作，这样大量的请求都会处于等待状态，很不利于生产开发。

```java
public static void main(String[] args) throws IOException {
    //创建一个服务器
    ServerSocket serverSocket = new ServerSocket( port: 8989);
    System.out.println("服务器已经启动........");
    while(true) {
        //让服务器接受|接收客户端
        Socket socket = serverSocket.accept();

        //处理请求数据
        InputStream inputStream = socket.getInputStream();
        StringBuilder builder = new StringBuilder();
        int len = 0;
        byte[] b = new byte[1024];
        while (true) {
            len = inputStream.read(b);
            if (len == -1) {break;}
            builder.append(new String(b,  offset: 0, len));
        }

        System.out.println(builder.toString());
        socket.shutdownInput();//获取请求数据 结束
        //处理业务
        //服务端响应客户端数据
        OutputStream outputStream = socket.getOutputStream();
        outputStream.write("讲".getBytes());
        socket.shutdownOutput();
    }
```

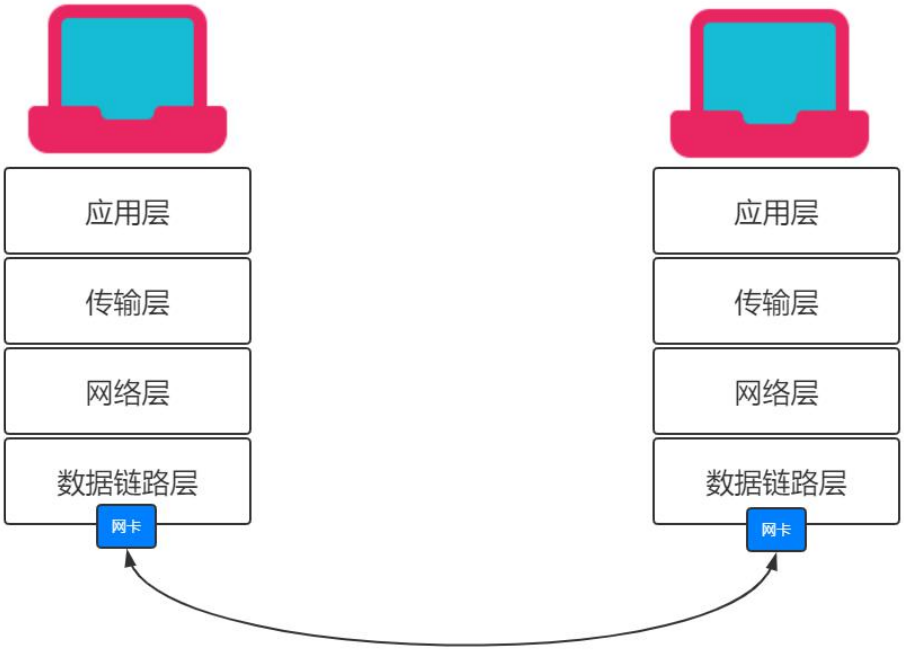
获取的套接

操作

必须等套接字操作完，释放掉，才能捕获下一个套接字

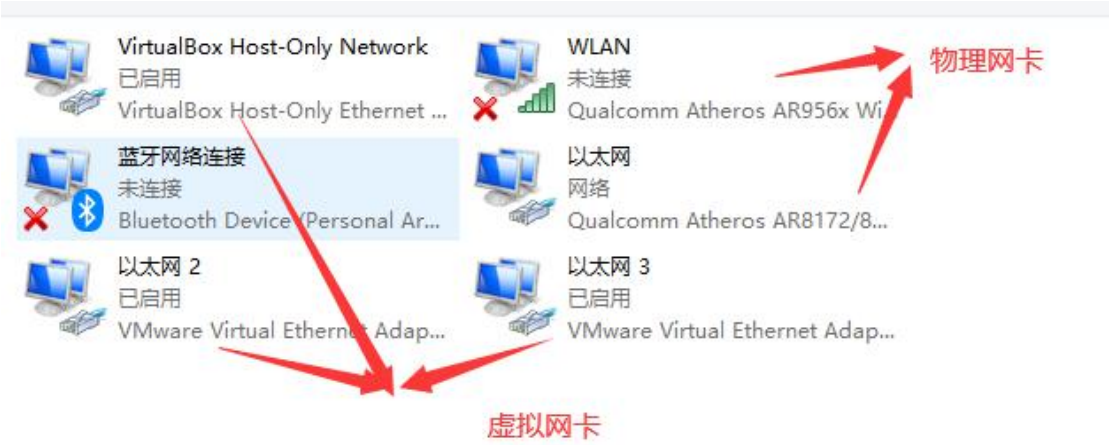
二、 网卡和 IP 详解

1．网卡



操作系统默认使用物理网卡，每一个物理网卡都有唯一的 MAC 地址，也可以通过机制创建多张虚拟网卡，如在创建虚拟机、安装 Docker 时，操作系统就会为每一个虚拟机创建专属虚拟网卡，虚拟网卡依通过桥接物理网卡等形式工作。

每个网卡可以配置多个 IP，每个 IP 的端口上限是 65535



2．IP、网段、掩码

（1）IP

用来标记网络的唯一性标志，是 32 位的二进制数，分 4 段，在日常使用中以 10 进制表示；

（2）子网掩码

区分 IP 地址中的网络部分和主机部分，网络部分即常说同一个子网等

同样是 32 位的二进制数，分 4 段，在日常使用中以 10 进制表示

与 IP 进行与运算，如果相同则表示处于同一个网络部分，即网段

常见的子网掩码，将网络部分标记为 255，将主机部分标记为 0，便于区分

掩码	IP	网络位	表示
255.0.0.0	192.168.1.1	192	192.168.1.1/8
255.255.0.0	192.168.1.1	192.168	192.168.1.1/16
255.255.255.0	192.168.1.1	192.168.1	192.168.1.1/24

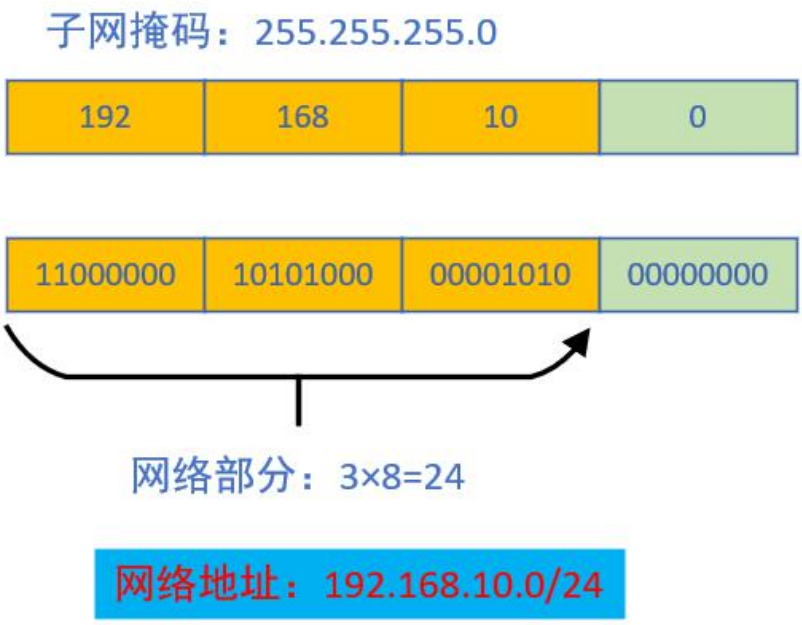
(3) 网段

网络部分一样的 IP 地址，称之为同一个网段

(4) 网络地址

在一个网段中最小的 IP 地址，即网络部分相同，主机部分为 0

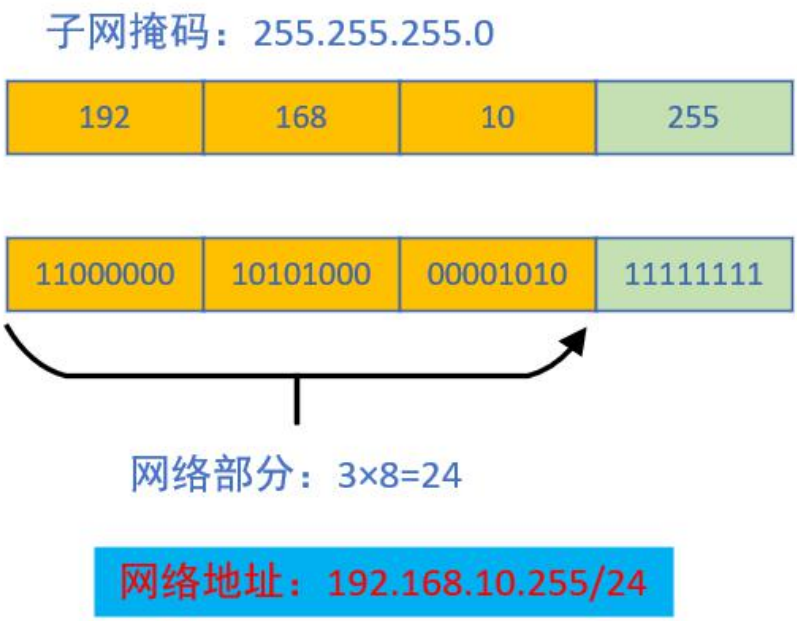
通常后面还要写明网络部分在二进制表示时的位数，为 8 的整数倍



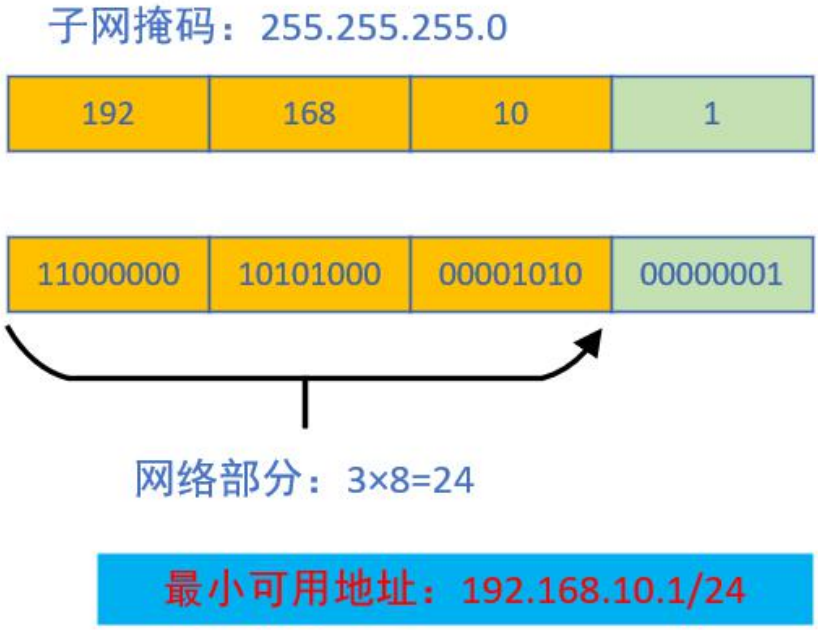
(5) 广播地址

在一个网段中最大的 IP 地址，即网络部分相同，主机部分为 255

通常后面还要写明网络部分在二进制表示时的位数，为 8 的整数倍



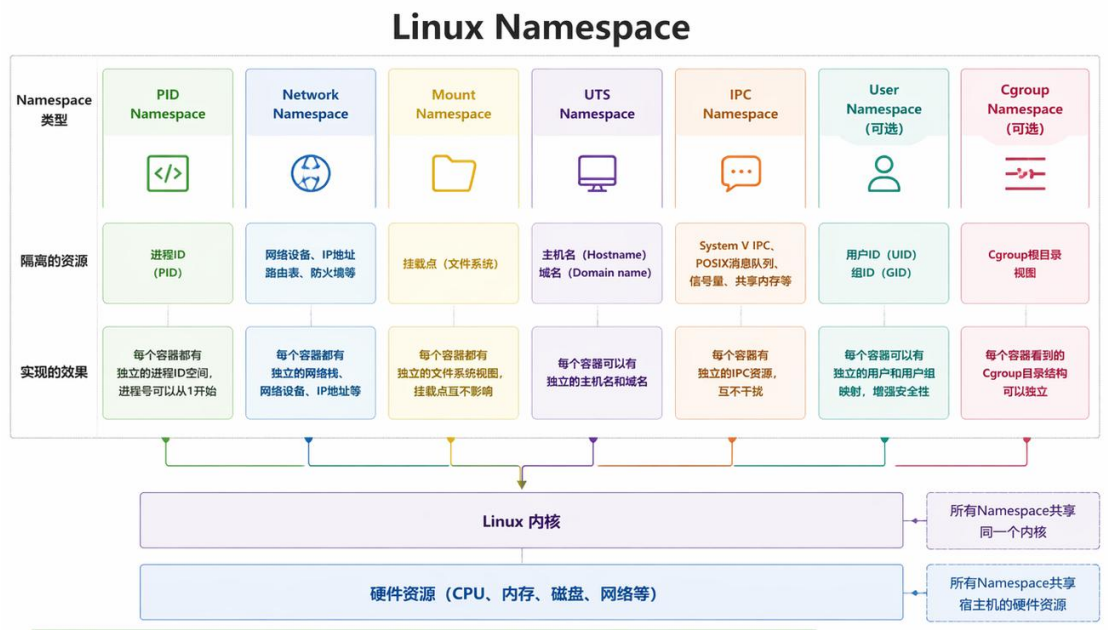
网络地址和广播地址不可以作为 IP，可用 IP 必须位于网络地址和广播地址的开区间内



三、 Linux 网络知识

1．Namespace

Linux Namespace 是 Linux 提供的一种内核级别环境隔离的方法。可以用来隔离网络、进程、文件系统等。它们相互独立但共享同一物理硬件资源（如 CPU、内存、磁盘）。

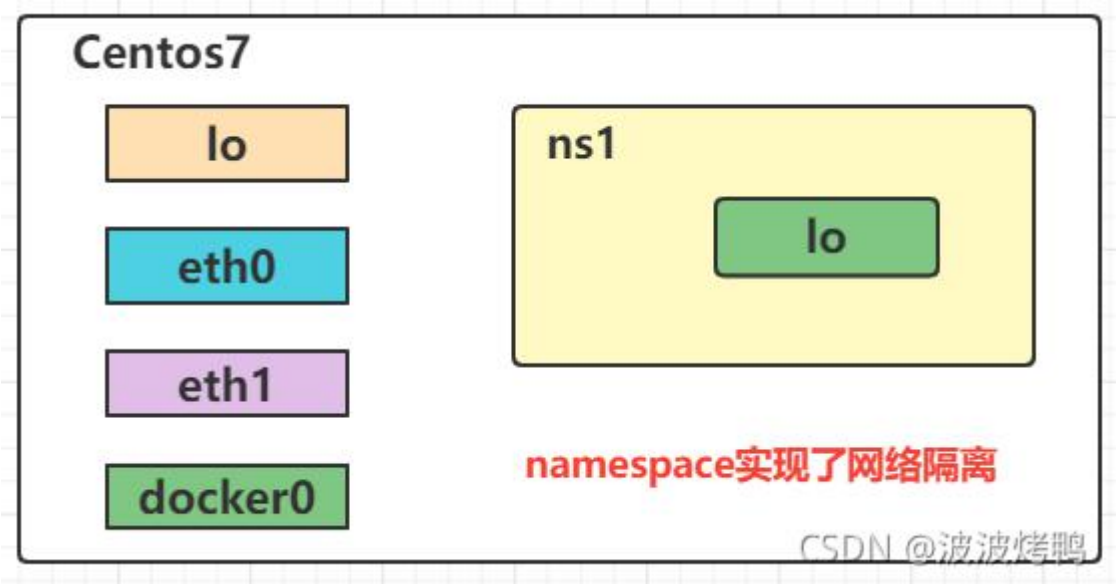


2．网络隔离（Network NameSpace）

（1）概念

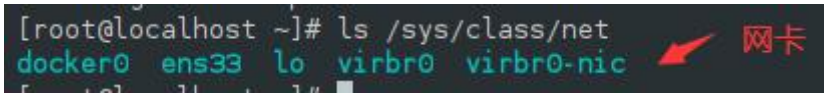
Network NameSpace（网络隔离技术）是 Linux 的 NameSpace 技术（资源隔离技术）之一。可以创建独立的网络空间，内置多张网卡，以文件的形式映射在磁盘中；

NameSpace 之间是相互隔离的，它们的网卡是无法通信的，相当于两个独立的局域网；



（2）默认情况

在 Linux 中，网卡运行在默认网络命名空间(Root Network Namespace)中，以文件的形式映射在默认文件夹



网卡状态有三个：UNKNOW、UP（开启）、DOWN（关闭），默认情况下是 DOWN(关闭).

```
[root@localhost ~]# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 00:0c:29:29:b2:d9 brd ff:ff:ff:ff:ff:ff
    inet 192.168.124.128/24 brd 192.168.124.255 scope global noprefixroute dynamic ens33
       valid_lft 1200sec preferred_lft 1200sec
    inet6 fe80::4cc8:8d14:2759:71ae/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: virbr0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default qlen 1000
    link/ether 52:54:00:b1:82:52 brd ff:ff:ff:ff:ff:ff
```

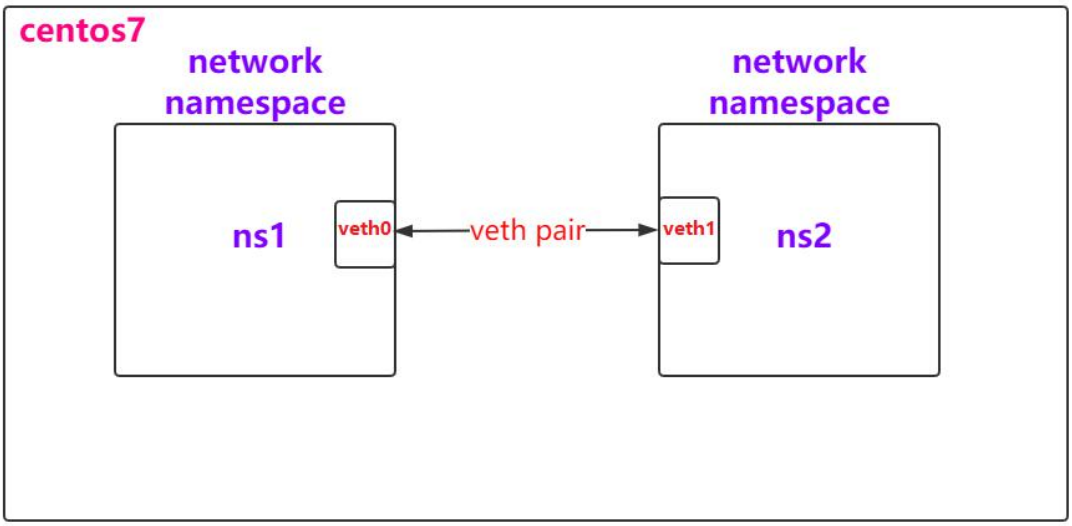
（3）自定义网络空间

可以通过命令创建自定义网络空间，添加网卡等设备，并为每一个网卡设置 IP 等信息；

```
[root@localhost ens33]# ip netns list
[root@localhost ens33]# ip netns add ns1
[root@localhost ens33]# ip netns exec ns1 ip a
1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
[root@localhost ens33]# ip netns exec ns1 ip link set lo up
[root@localhost ens33]# ip netns add ns2
[root@localhost ens33]#
```

(4) veth pair(Virtual Ethernet Pair)

veth pair 相当于一根虚拟网线，将两个 Network NameSpace 进行连接，并会分别在两端的网络空间创建一张虚拟网卡；

我们需要给这两张虚拟网卡设置同网段且不冲突的 IP，两个网络空间才能访问；

需要注意的是：Network NameSpace 通过 veth pair 连接，只能是点对点，两两互相，从一个网络空间到另一个；